

FFT DSP + génération PNG automatique + CI/CD Azure DevOps

Sommaire

1 Présentation.....	1
2 Objectifs.....	2
3 Créer la structure.....	2
4 Créer l'application console.....	2
5 installer les packages.....	3
6 compilation – phase de Build.....	3
7 Run – Phase d'Exécution.....	6
8 Résultat – fichier .png.....	7
8.1 fichiers .png.....	7
8.2 Courbe de densité spectrale de puissance (signal sinusoidal).....	7
8.3 Courbe de densité spectrale de puissance (signal gaussien).....	8
9 git et repo GitHub.....	8
10 Azure DevOps.....	11
10.1 Créer un nouveau projet.....	11
10.3 Résultats.....	12
10.3.1 État du pipeline :.....	12
10.3.2 Artifacts publiés :.....	13
11 Références.....	14

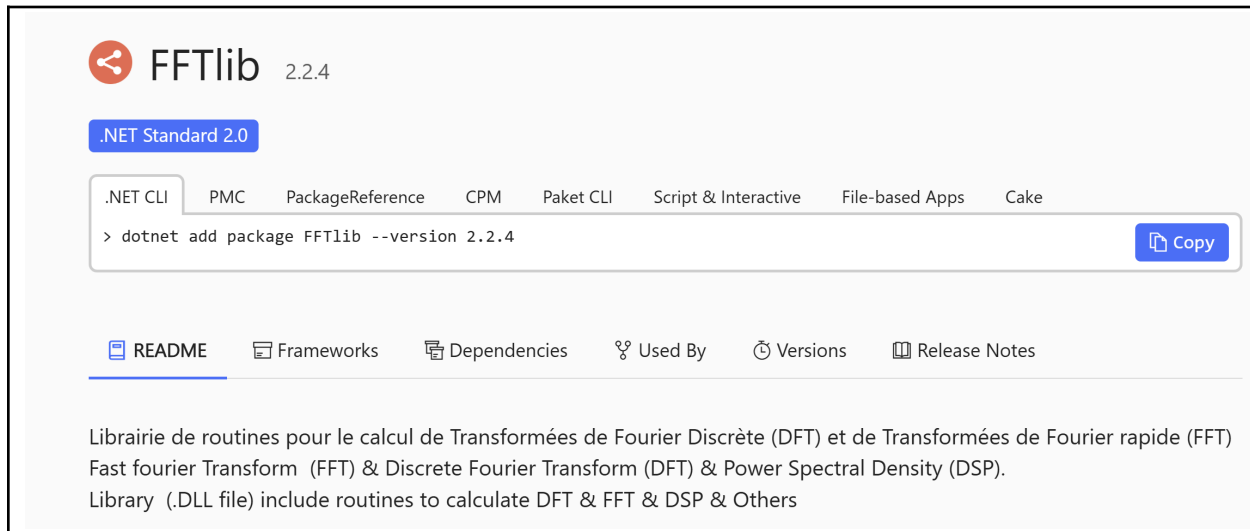
1 Présentation

This lab demonstrates the implementation of a complete CI/CD pipeline using Azure DevOps for a custom C# FFT (Fast Fourier Transform) library published on NuGet.

The program generate DSP of test signals, and plot graphs for the DSP. The graphs are saved in .png files

The project uses the custom package `FFTlib` developed and published by the author ; Henri Guillot instead of relying on a third-party FFT framework.

Link : <https://www.nuget.org/packages/Math-FFT-Lib>



The lab focuses on practical DevOps workflows for .NET applications including automated builds, package restoration from NuGet, unit testing, artifact publishing and GitHub integration.

2 Objectifs

- Build a .NET 8 C# console application using a custom FFT NuGet package
- Integrate GitHub with Azure DevOps
- Create a YAML-based CI/CD pipeline
- Automate build and test execution
- Publish build artifacts
- Demonstrate real-world package consumption from NuGet
- Reinforce practical Azure DevOps pipeline concepts

Installation du package FFTLib :

```
dotnet add package FFTlib --version 2.2.4
```

3 Créer la structure

```
Mkdir fft-plotlib-local-test
```

```
Cd /fft-plotlib-local-test
```

```
mkdir src
```

```
mkdir tests
```

```
henri@Henri:/mnt/c/temp/labs/fft-plotlib-local-test$ tree -L 1
.
├── Program-cs.backup
├── azure-pipelines.yml
├── src
└── tests

3 directories, 2 files
henri@Henri:/mnt/c/temp/labs/fft-plotlib-local-test$
```

This PC > Windows (C:) > temp > labs > fft-plotlib-local-test >				
Name	Date modified	Type	Size	
src	5/20/2026 8:55 PM	File folder		
tests	5/20/2026 8:54 PM	File folder		
.gitignore	5/21/2026 10:54 AM	Git Ignore Source ...	1 KB	
azure-pipelines.yml	5/21/2026 11:00 AM	YML File	0 KB	
Program-cs.backup	5/21/2026 10:21 AM	BACKUP File	3 KB	

4 Créer l'application console

```
cd src
dotnet new console -n ConsoleTest
```

5 installer les packages

```
Cd ConsoleTest
dotnet add package FFTlib --version 2.2.4
dotnet add package ScottPlot
```

6 compilation – phase de Build

Ouvrir program.cs dans src/ConsoleTest

Ajouter le code suivant : dans le fichier : program.cs

```

using System;
using System.Numerics;
using FFTlib;
using ScottPlot;
using System.IO;

namespace Test9
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=====");
            Console.WriteLine("FFT / DSP Demonstration");
            Console.WriteLine("=====");

            int nbechant = 256;

            Directory.CreateDirectory("output");

            // =====
            // SIGNAL 1 : PURE SINUSOID
            // =====

            Console.WriteLine("Generating sinusoidal signal...");

            decimal[] signal1 = SiTest.GenSin2piF(
                nbechant,
                1,
                50,
                1000
            );

            Complex[] Ts1 = DFT.DFTv2(signal1);

            decimal[] Module1 = DSP.Dspdeci(Ts1);

            double[] y1 = Array.ConvertAll(
                Module1,
                d => (double)d
            );

            int half1 = y1.Length / 2;

            double[] x1 = new double[half1];
            double[] y1half = new double[half1];

```

```

for (int i = 0; i < half1; i++)
{
    x1[i] = i;
    y1half[i] = y1[i];
}

var plt1 = new ScottPlot.Plot();

plt1.Add.Scatter(x1, y1half);

plt1.Title("DSP - Pure Sinusoidal Signal");

plt1.XLabel("Frequency Bin");

plt1.YLabel("Amplitude");

plt1.SavePng("output/fft-sinus.png", 1200, 600);

Console.WriteLine("PNG generated: fft-sinus.png");

// =====
// SIGNAL 2 : GAUSSIAN / RANDOM SIGNAL
// =====

Console.WriteLine("Generating random gaussian-like signal...");

Random rnd = new Random();

decimal[] signal2 = new decimal[nbechant];

for (int i = 0; i < nbechant; i++)
{
    // random centered noise
    double noise =
        (rnd.NextDouble() - 0.5) +
        (rnd.NextDouble() - 0.5) +
        (rnd.NextDouble() - 0.5);

    signal2[i] = (decimal)noise;
}

Complex[] Ts2 = DFT.DFTv2(signal2);

decimal[] Module2 = DSP.Dspdeci(Ts2);

double[] y2 = Array.ConvertAll(
    Module2,
    d => (double)d

```

```

);

int half2 = y2.Length / 2;

double[] x2 = new double[half2];
double[] y2half = new double[half2];

for (int i = 0; i < half2; i++)
{
    x2[i] = i;
    y2half[i] = y2[i];
}

var plt2 = new ScottPlot.Plot();

plt2.Add.Scatter(x2, y2half);

plt2.Title("DSP - Gaussian / Random Signal");

plt2.XLabel("Frequency Bin");

plt2.YLabel("Amplitude");

plt2.SavePng("output/fft-gaussian.png", 1200, 600);

Console.WriteLine("PNG generated: fft-gaussian.png");

Console.WriteLine("=====");
Console.WriteLine("Processing complete.");
Console.WriteLine("=====");
}
}

}

```

Aller a la racine du projet :

Cd src/consoleTest

```

PS C:\temp\labs\fft-plotlib-local-test\src> cd ..
PS C:\temp\labs\fft-plotlib-local-test> cd src\ConsoleTest
PS C:\temp\labs\fft-plotlib-local-test\src\ConsoleTest> pwd

Path
----
C:\temp\labs\fft-plotlib-local-test\src\ConsoleTest

```

Puis

Dotnet build

```

PS C:\temp\labs\fft-plotlib-local-test\src\ConsoleTest> dotnet build
Determining projects to restore...
All projects are up-to-date for restore.
ConsoleTest -> C:\temp\labs\fft-plotlib-local-test\src\ConsoleTest\bin\Debug\net8.0\ConsoleTest.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:01.05

```

7 Run – Phase d'Exécution

Dotnet run

```

PS C:\temp\labs\fft-plotlib-local-test\src\consoleTest> dotnet run
=====
FFT / DSP Demonstration
=====
Generating sinusoidal signal...
PNG generated: fft-sinus.png
Generating random gaussian-like signal...
PNG generated: fft-gaussian.png
=====
Processing complete.
=====
PS C:\temp\labs\fft-plotlib-local-test\src\consoleTest>

```

8 Résultat – fichier .png

8.1 fichiers .png

Les fichiers résultats sont : fft-sinus.png et fft-gaussian.png

Ils représentent la densité spectrale de puissance des signaux de test.

```

PS C:\temp\labs\fft-plotlib-local-test\src\ConsoleTest\output> dir

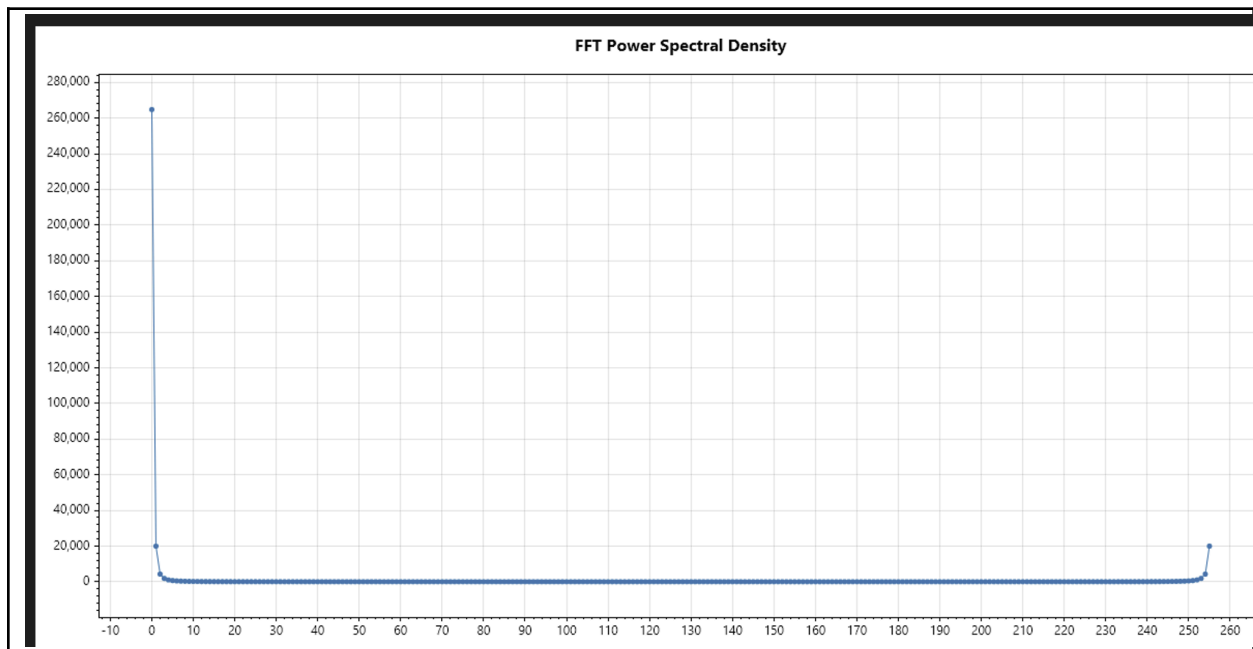
Directory: C:\temp\labs\fft-plotlib-local-test\src\ConsoleTest\output

Mode                LastWriteTime         Length Name
----                -
-a----             5/21/2026  11:09 AM         86582 fft-gaussian.png
-a----             5/21/2026  11:09 AM         27809 fft-sinus.png
PS C:\temp\labs\fft-plotlib-local-test\src\ConsoleTest\output>

```

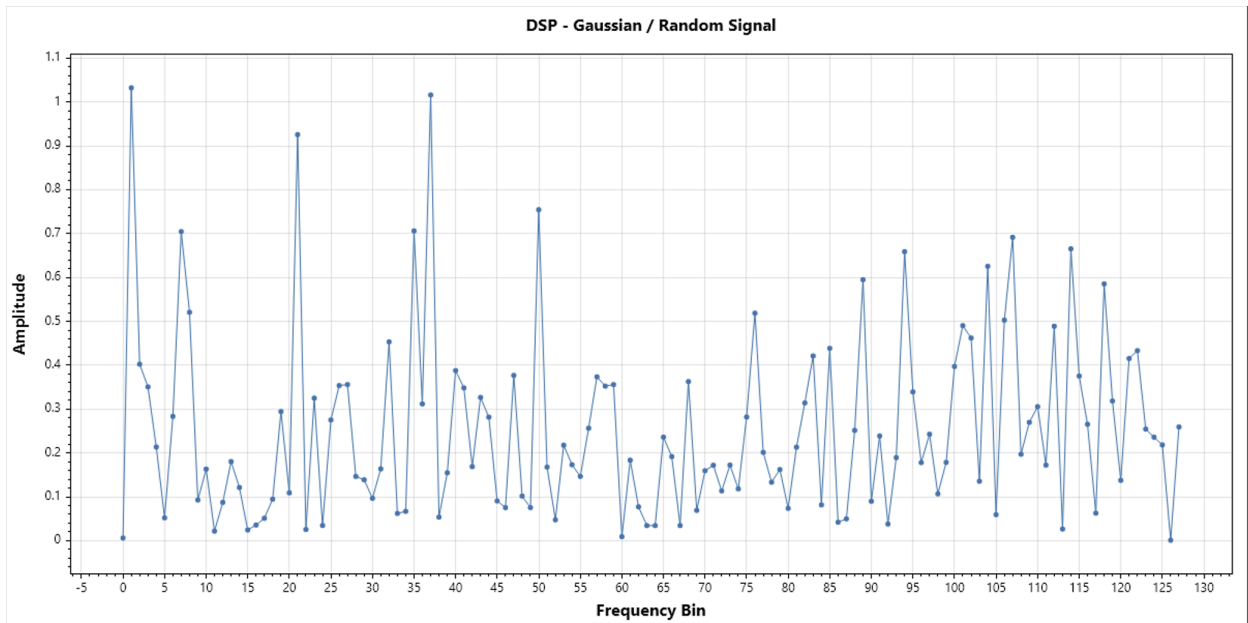
8.2 Courbe de densité spectrale de puissance (signal sinusoidal)

Fichier : fft-sinus.png



8.3 Courbe de densité spectrale de puissance (signal gaussien)

Fichier : fft-gaussian.png



9 git et repo GitHub

Revenir a la racine du projet :

```
cd ../../
```

initialiser git

```
git init
```

créer le fichier : .gitignore

```
bin/  
obj/  
.vs/
```

Ajouter les fichiers

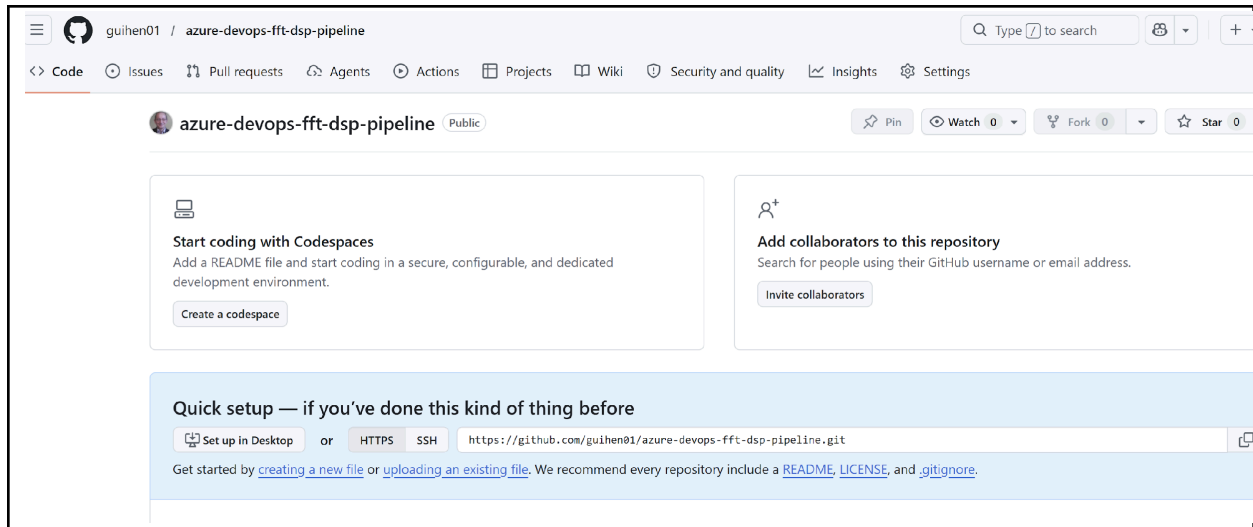
```
git add .
```

faire un premier commit :

```
git commit -m "Initial commit - Custom FFTLib Azure DevOps lab"
```

Créer le repo GitHub

Aller sur GitHub et créer le repo : azure-devops-fft-dsp-pipeline :



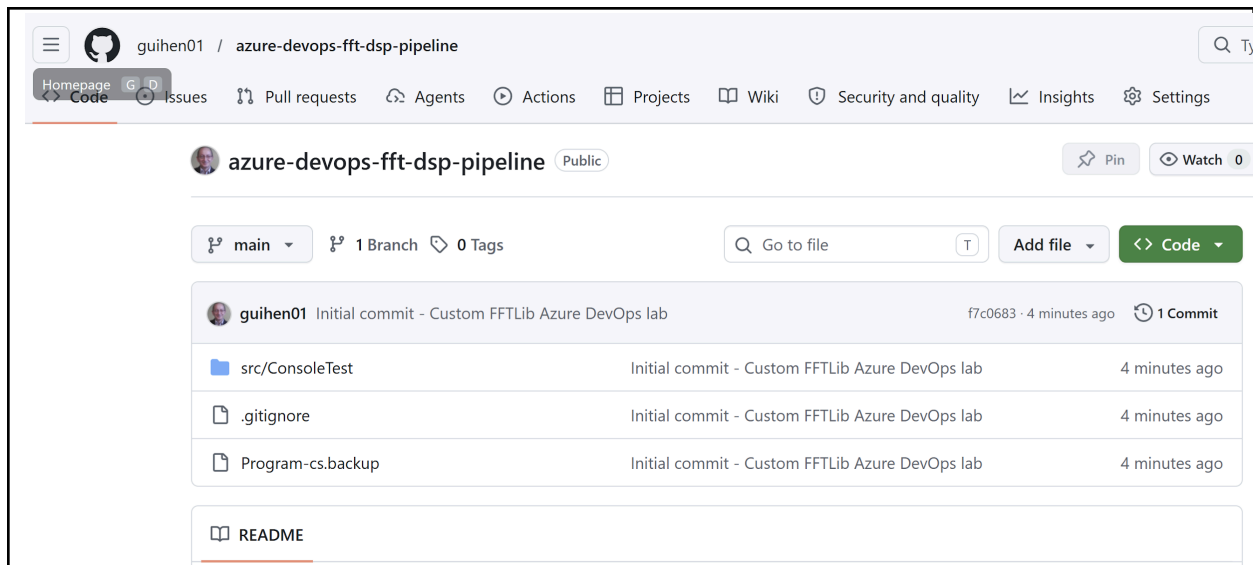
Saisir la commande (fournie et affichée par git sur la page initiale du repo créée) :

```
git remote add origin
https://github.com/guihen01/azure-devops-fft-dsp-pipeline.git
```

Faire un push :

```
git branch -M main
git push -u origin main
```

Résultat :



Créer le pipeline : azure-pipelines.yml
Se positionner à la racine du projet

Mettre le code ci-dessous dans le fichier : azure-pipelines.yml

```
trigger:
- main

pool:
  vmImage: 'ubuntu-latest'

steps:

# =====
# Install .NET SDK
# =====

- task: UseDotNet@2
  inputs:
    packageType: 'sdk'
    version: '8.x'

# =====
# Restore NuGet packages
# =====

- script: |
  dotnet restore src/ConsoleTest/ConsoleTest.csproj
  displayName: 'Restore Packages'

# =====
# Build application
# =====

- script: |
  dotnet build src/ConsoleTest/ConsoleTest.csproj --configuration Release
  displayName: 'Build Application'

# =====
# Run FFT DSP application
# Generates PNG files
# =====

- script: |
  dotnet run --project src/ConsoleTest/ConsoleTest.csproj
  displayName: 'Run FFT DSP Application'

# =====
# Publish generated FFT PNG images
# =====
```

```

- task: PublishBuildArtifacts@1
  inputs:
    PathtoPublish: 'src/ConsoleTest/output'
    ArtifactName: 'fft-images'

# =====
# Publish application artifacts
# =====

- script: |
  dotnet publish src/ConsoleTest/ConsoleTest.csproj -c Release -o $(Build.ArtifactStagingDirectory)
  displayName: 'Publish Application'

- task: PublishBuildArtifacts@1
  inputs:
    PathtoPublish: '$(Build.ArtifactStagingDirectory)'
    ArtifactName: 'fft-console-app'

```

Faire un commit du pipeline
Cela poussera le fichier vers le repository github

```

git add .
git commit -m "Add Azure DevOps pipeline"
git push

```

10 Azure DevOps

Aller dans Azure DevOps

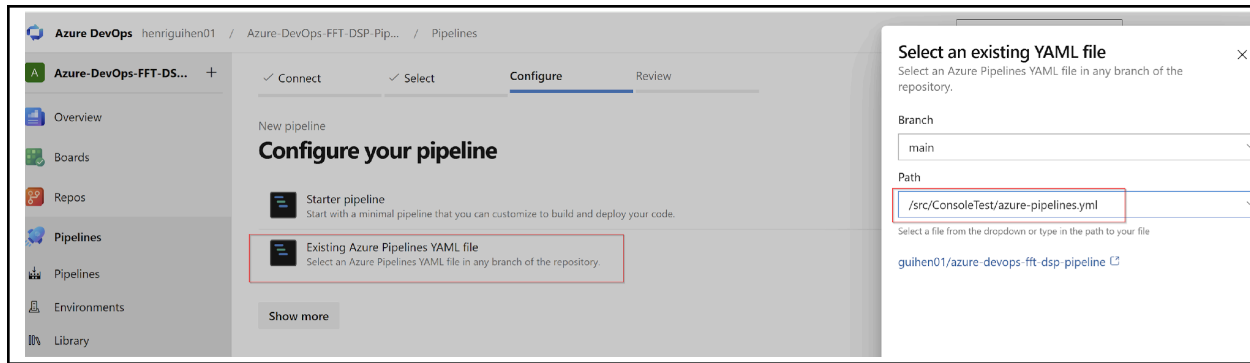
10.1 Créer un nouveau projet

Nom du projet : Azure-DevOps-FFT-DSP-Pipeline

10.2 Créer le pipeline :

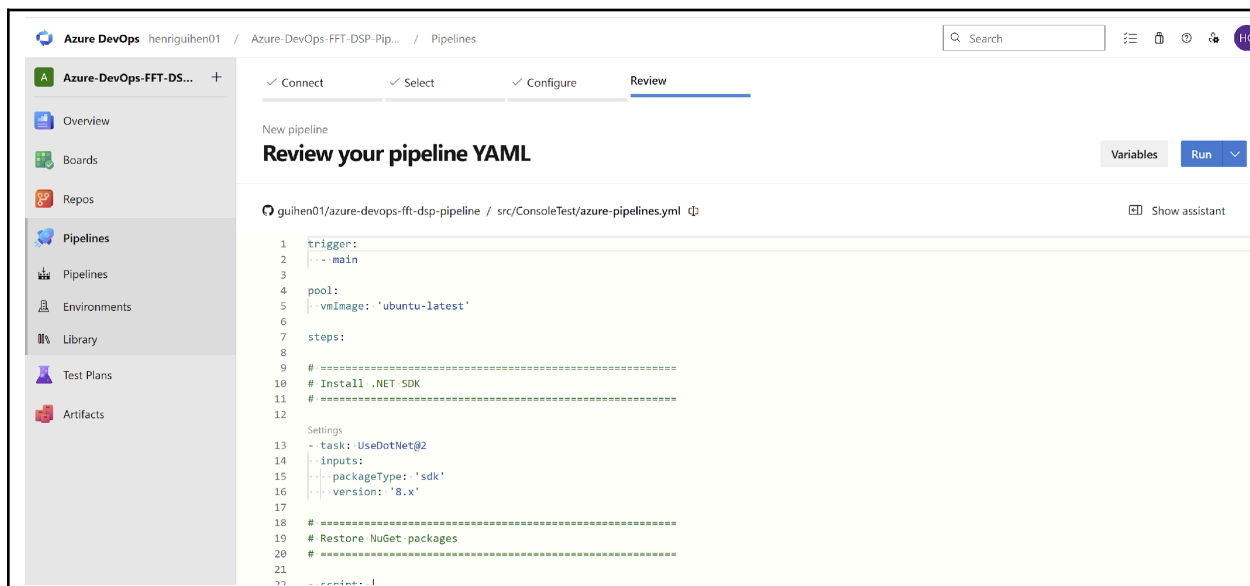
Pipelines

- New Pipeline
- GitHub
- sélectionner repo : azure-devops-fft-dsp-pipeline
- Existing Azure Pipelines YAML file



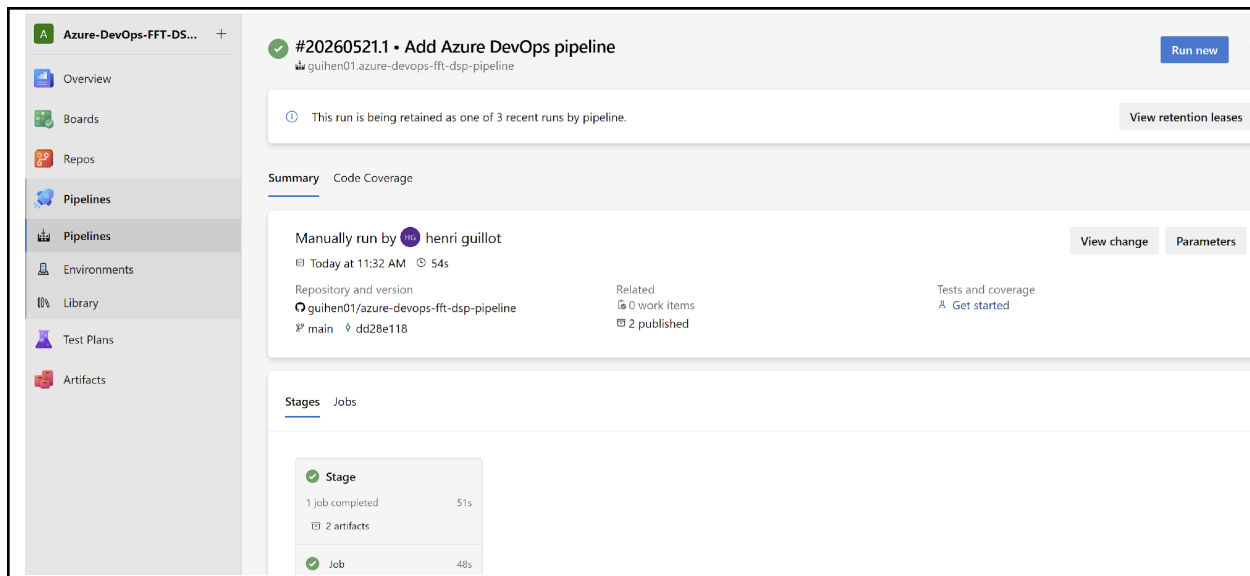
Choisir : azure-pipelines.yml

Cliquer sur : run pipeline



10.3 Résultats

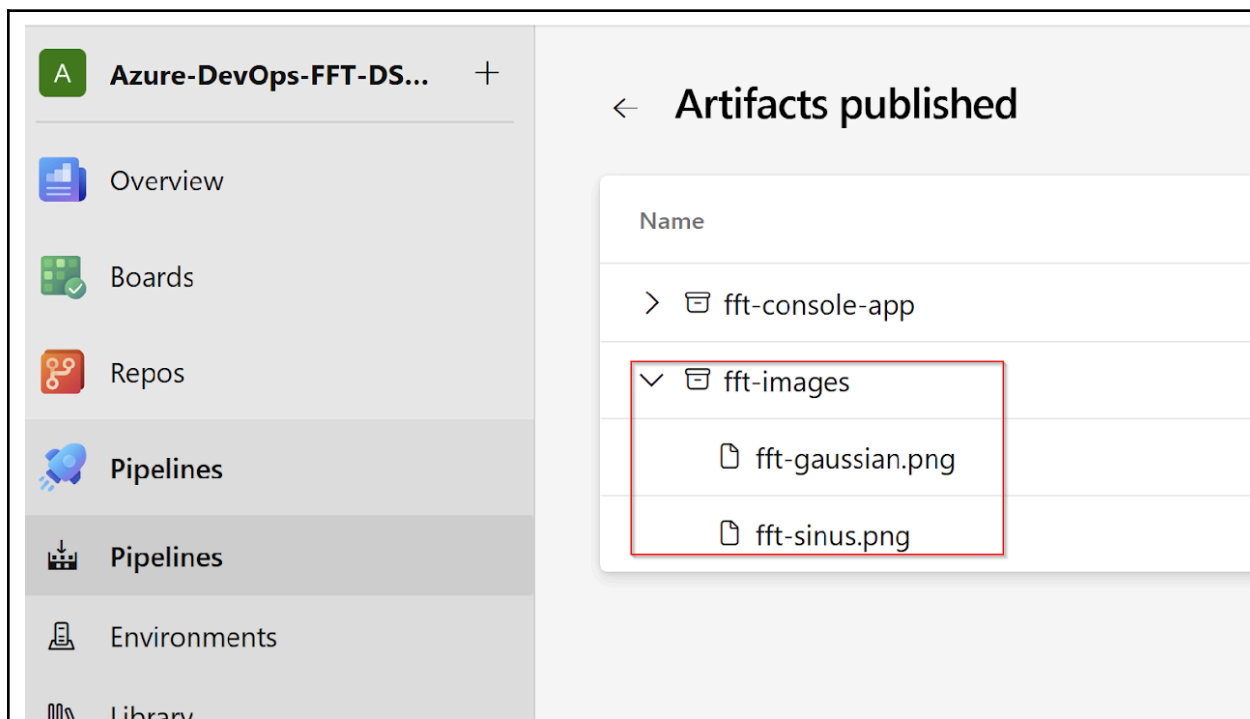
10.3.1 État du pipeline :



10.3.2 Artifacts publiés :

On voit les artifacts publiés, ce sont deux fichiers .png. qui montrent les courbes de densité spectrale de puissance des signaux de test.

On peut cliquer sur les fichiers et les télécharger



11 Références

NuGet package:

<https://www.nuget.org/packages/FFTlib/>

GitHub repository:

<https://github.com/guihen01/Math-FFT-Lib>